SCHOOL OF TECHNOLOGY	WOXSEN UNIVERSITY

DATABASE MANAGEMENT SYSTEMS
PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

Design and Implementation of a Database Management System
for E-Commerce Order, Return and Refund Management

Program	B.Tech – Artificial Intelligence & Machine Learning, School of Technology
Section	A
Faculty	DBMS Course Faculty, School of Technology, Woxsen University
Date of Submission	10 October 2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102114	Kakaraparthi Komal Prajvesh		

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, Name of the Faculty, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled “Design and Implementation of a Database Management System for E-Commerce Order, Return and Refund Management”, submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been given. The MySQL 8 documentation, FastAPI documentation, and standard DBMS textbooks consulted are cited in the References [5]-[8].

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date :

Place :

Name:
Kakaraparthi
Komal Prajvesh

Roll Number:
25WU0102114

Program: B.Tech
AI & ML

TABLE OF CONTENTS

Use this common structure for all DBMS PBL projects. Enter or update the page numbers after completing the report.

Contents	Page No.
Abstract	
LIST OF TABLES	
LIST OF FIGURES	
LIST OF ACRONYMS	
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	
1.1 Problem Statement	
1.2 Objectives	
1.3 Scope and Assumptions	
1.4 Proposed Solution Strategy	
1.5 Functional Requirements	
1.6 Entity-Relationship Diagram (ERD)	
1.7 Entity and Relationship Summary	
2. LOGICAL DATABASE DESIGN	
2.1 Relational Schema	
2.2 Keys and Integrity Constraints	
2.3 Functional Dependencies	
2.4 Normalization up to Third Normal Form (3NF)	
2.5 Data Dictionary	
2.6 Implementation of Logical Design: DDL and Sample Data	
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	
3.1 Application Overview	
3.2 Development Environment and Database Connectivity	
3.3 Application Modules and Database Mapping	
3.4 CRUD Operations	
3.5 Business Rules, Validation and Error Handling	
3.6 Reports and Database Operations	
3.7 Application Screens	
3.8 Testing and Results	
4. CONCLUSION	
4.1 Limitations and Future Enhancements	
5. REFERENCES	

Contents	Page No.
6. APPENDICES	

LIST OF TABLES

Update this list after finalising the report. Use Word References > Insert Caption for every table and References > Insert Table of Figures with the caption label "Table".

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	
Table 2	Proposed Solution Strategy	
Table 3	Functional Requirements	
Table 4	Entity and Relationship Summary	
Table 5	Relational Schema	
Table 6	Keys and Integrity Constraints	
Table 7	Normalization Status up to 3NF	
Table 8	Data Dictionary	
Table 9	Database Implementation Environment	
Table 10	Application Development Environment	
Table 11	Application Module-to-Database Mapping	
Table 12	CRUD Operations	
Table 13	Business Rules, Validation and Error Handling	
Table 14	Reports and Database Operations	
Table 15	Test Cases	

LIST OF FIGURES

Update this list after finalising the report. Use Word References > Insert Caption for every figure and References > Insert Table of Figures with the caption label "Figure".

Figure No.	Figure Title	Page No.
Figure 1	Streamlit Customer Portal	
Figure 2	Streamlit Operations Dashboard	
Figure 3	Streamlit Analytics Dashboard	
Figure 4	Streamlit Admin Audit Log	
Figure 5	Admin Audit Log — immutable trail	
Figure 6	Return lifecycle state machine	
Figure 7	Three-layer system architecture	
Figure 8	Automated test evidence (13/13 passed)	
Figure 9	Performance benchmark chart	
Figure 10	Late-return trigger block evidence	

LIST OF ACRONYMS

Retain only the acronyms used in the report and add any project-specific acronyms. All the Acronyms to be sorted order.

Acronym	Expansion
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
DBMS	Database Management System
DDL / DML	Data Definition Language / Data Manipulation Language
ENUM	Enumerated (restricted status domain type in MySQL)
ERD	Entity-Relationship Diagram
FK / PK	Foreign Key / Primary Key
JWT	JSON Web Token
RBAC	Role-Based Access Control
SPA	Single Page Application
SQL	Structured Query Language
PBL	Project-Based Learning
SRS	Software Requirements Specification

ABSTRACT

Write the abstract as one self-contained paragraph of about 150-200 words. Do not use bullets, tables, screenshots, citations or code in the abstract.

Part of Abstract	What to Write
Problem / Context	State the real-world problem and why a database-based solution is needed (2-3 sentences).
Objective	State the main objective of the project in one clear sentence.
Solution Approach	Briefly mention the ER model, logical database design/normalization, DBMS used, and Java/Python application approach.
Major Functions / Results	Mention the main application functions, important database operations/reports, and what the completed system demonstrates.
Closing Statement	Conclude with the practical outcome or usefulness of the developed application.

E-commerce post-purchase operations routinely fail when orders, returns, and refunds are tracked in disconnected spreadsheets, causing untracked status changes, duplicate refunds, and missing audit evidence. The objective of this project is to design and implement a centralized ACID-compliant DBMS for the complete order-to-refund lifecycle. The solution uses a 13-table Third Normal Form (3NF) MySQL 8 (InnoDB) schema covering customers, products, inventory, orders, payments, item-level returns, warehouse inspection, refunds, and an immutable audit log; five triggers enforce return-window eligibility, return-quantity limits, strict state transitions, and automatic auditing; and one stored procedure, sp_process_refund, executes atomic refund processing with duplicate-refund prevention and full rollback on error [5], [7]. A role-based Streamlit application connects directly to the database with parameterized SQL to implement ordering, returning, approving, receiving, inspecting, refunding, auditing, and analytics, while the FastAPI service is retained as the automated test and benchmark harness [6], [9]. The completed system demonstrates the full lifecycle Order → Requested → Approved → Received → Inspected → Refunded, passes 13/13 automated tests, sustains sub-millisecond indexed lookups at 1K scale, and verifies backup/restore integrity at 111,025 orders. The application provides a single queryable source of truth for post-purchase operations with enforced business rules and traceable refunds.

KEYWORDS

Provide 4-6 specific keywords separated by commas. Include the application domain and important technical terms. Avoid complete sentences and overly general words.

Keywords: E-Commerce Returns, Streamlit, MySQL, 3NF Normalization, Stored Procedures, Triggers

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

This chapter should represent the evidence presented in Review-I. Keep the discussion specific to the allotted project scenario.

1.1 Problem Statement

Describe the current problem, affected users/stakeholders, limitations of the existing/manual process, and the need for the proposed database application.

E-commerce after-sales handling is typically scattered across order spreadsheets, chat logs, and manual refund sheets. Support staff cannot see one dependable status for an order, warehouse staff record inspection results separately from the return approval, and finance has no systematic guard against duplicate refunds or late returns outside the product return window. Customers of stores such as the demo catalogue (headphones, keyboards, monitors, watches, docking stations) experience delayed or inconsistent refunds, while the business cannot answer analytical questions such as return rate per product or monthly refund totals. This project builds a single relational source of truth that links Customer → Orders → OrderItem → Payment → ReturnRequest → ReturnItem → Inspection → Refund, with every status change written to an immutable AuditLog. Database-enforced triggers block late returns, excess quantities, and invalid state jumps, and a transactional stored procedure guarantees that a refund is calculated from historical purchase prices exactly once or not at all. Centralizing such workflows in a relational DBMS with enforced integrity is the standard remedy for update anomalies and inconsistent reporting [1], [4].

1.2 Objectives

State 3-5 measurable objectives beginning with action verbs such as design, develop, implement, validate, automate, track or generate.

- Design a 13-entity ER model and implement it as a normalized 3NF MySQL 8 (InnoDB) schema with PK/FK, CHECK, UNIQUE, ENUM, and ON DELETE RESTRICT integrity.

- Implement 5 triggers and 1 transactional stored procedure (sp_process_refund) to automate return eligibility, state-machine enforcement, audit logging, and atomic refunds.
- Develop a role-based Streamlit application with direct parameterized-SQL connectivity and session-state RBAC for Customer, Support, Warehouse, and Admin, covering ordering, returns, inspection, refunds, analytics, and audit, while retaining FastAPI as the automated test/benchmark harness.
- Validate the system with 13 automated tests (100% pass) covering constraints, procedure logic, full lifecycle integration, rollback, and concurrent duplicate-refund handling.
- Generate operational and analytical reports (order history, pending returns, refund status, return rates, monthly refunds, top reasons) and verify performance up to 100K orders with backup/restore checks.
- Validate the system with 13 automated tests (100% pass) covering constraints, procedure logic, full lifecycle integration, rollback, and concurrent duplicate-refund handling.

1.3 Scope and Assumptions

Table 1. Scope and Assumptions

Item	Description
In Scope	Order placement with mock payment; item-level return requests; support approve/reject; warehouse receive + inspection (GOOD/DAMAGED/DEFECTIVE); atomic refund via sp_process_refund; inventory restock on GOOD; immutable audit log; 6 operational/analytical reports; 4-role RBAC demo.
Out of Scope / Exclusions	Real bank/card network integration (mock gateway only); multi-region replication; image-based auto-inspection; SMS/email webhooks; mobile app; distributed sharding.
Assumptions	Single MySQL 8 InnoDB instance on localhost/Docker; all prices in one currency; return window counted from Orders.order_date per product return_window_days; demo users share password password123; tested to 100K orders locally.

1.4 Proposed Solution Strategy

Explain how the proposed solution will address the identified problem. Describe the intended users, major modules, workflow, key business rules and expected outputs.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Customer (places orders, requests returns); Support Agent (approves/rejects, triggers refunds); Warehouse Operator (receives, inspects); Admin (full access + audit log).
Major Application Modules	Streamlit pages: sidebar login + session-state RBAC; Customer Portal (catalogue, orders, returns); Operations (support approve/reject, warehouse receive/inspect); Refunds (sp_process_refund); Analytics (metrics, charts, 6 reports); Audit viewer (Admin); 7-state Lifecycle Demo stepper.
Proposed Workflow	Customer orders (Delivered) → requests return (Requested) → Support approves (Approved) → Warehouse receives (Received) → inspects (Inspected) → Support/Admin refunds (Refunded) via Streamlit pages; Rejected is terminal; triggers block any skip.
Important Business Rules	Return only within product window; qty <= purchased - already returned; strict state machine; one processed refund per return; GOOD restocks inventory; every change audited.
Expected Reports / Outputs	Order history; pending returns queue; refund status by order; return rate per product; monthly refund totals; top return reasons; Admin audit log.

1.5 Functional Requirements

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Customer places order with items; system prices from Product, creates Orders + OrderItem + Payment (mock	High

	gateway reference).	
FR-02	Customer submits item-level return request (reason + quantities) within return window.	High
FR-03	Support approves or rejects a Requested return (state trigger enforces).	High
FR-04	Warehouse marks Approved return as Received on physical arrival.	High
FR-05	Warehouse records Inspection (GOOD/DAMAGED/DEFECTIVE + notes); return becomes Inspected.	High
FR-06	Support/Admin processes refund exactly once via sp_process_refund (amount = Σ qty × unit_price_at_purchase); duplicate blocked, rollback on error.	High
FR-07	Admin views immutable AuditLog of all ReturnRequest/Refund status changes with actor and timestamp.	Medium
FR-08	All roles view analytics: order history, pending returns, refund status, return rates, monthly refunds, top reasons (RBAC-filtered endpoints).	Medium

1.6 Entity-Relationship Diagram (ERD)

Insert the approved ER diagram. It should show entities, major attributes, primary keys, relationships, cardinalities and participation constraints where applicable.

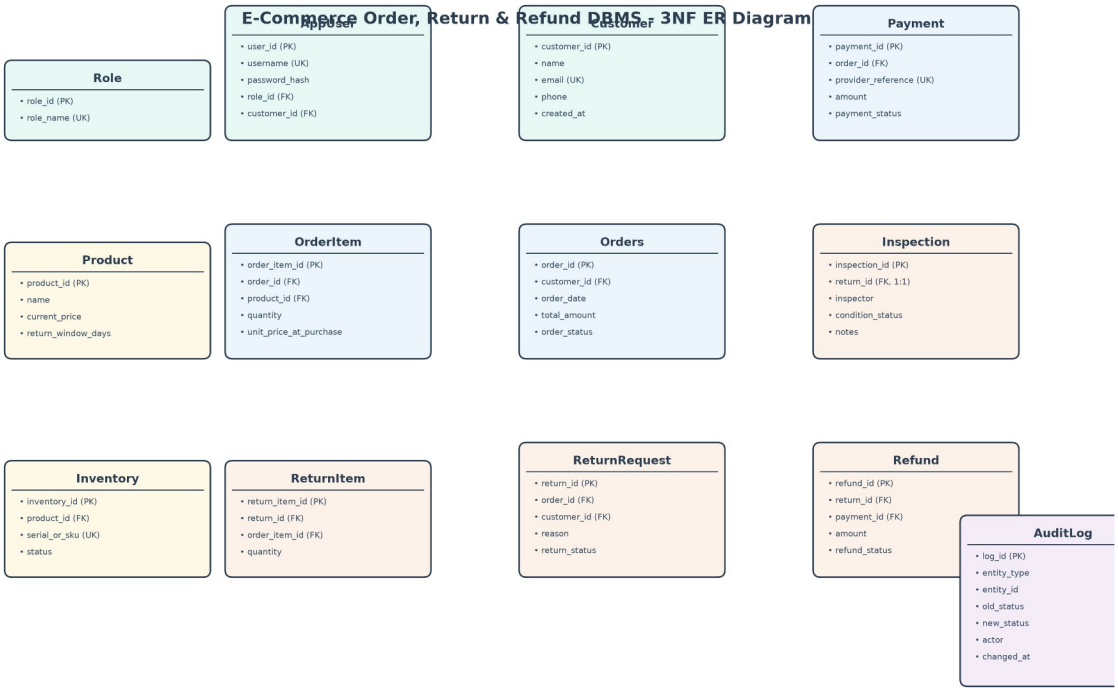


Figure 1. Entity-Relationship Diagram of the Proposed System (13 entities; 1:1 Inspection, 1:N Refund; item-level returns)

1.7 Entity and Relationship Summary

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
Role	role_id (PK), role_name (UK)	RBAC roles: Customer, Support, Warehouse, Admin.
Customer	customer_id (PK), email (UK)	Buyer profile, separate from login credentials.
AppUser	user_id (PK), username (UK); FK role_id, customer_id	Login credentials + role assignment; bcrypt hash.
Product	product_id (PK), current_price, return_window_days	Catalogue master data; window 7/14/30 days.
Inventory	inventory_id (PK), serial_or_sku (UK); FK product_id	Stock units; status AVAILABLE/RESERVED/ SOLD/RETURNED/DAMAGED.
Orders	order_id (PK); FK customer_id	Order header; total_amount; status Pending..Cancelled.
OrderItem	order_item_id (PK); FK	Line items; preserves

	order_id, product_id	unit_price_at_purchase (history).
Payment	payment_id (PK), provider_reference (UK); FK order_id	Mock gateway record; no raw card data stored.
ReturnRequest	return_id (PK); FK order_id, customer_id	Return header; 6-state lifecycle; see Fig. lifecycle.
ReturnItem	return_item_id (PK); FK return_id, order_item_id	Partial returns; qty > 0 enforced by trigger.
Inspection	inspection_id (PK), return_id (UK, 1:1); FK return_id	Warehouse verdict GOOD/DAMAGED/DEFECTIVE + inspector.
Refund	refund_id (PK); FK return_id, payment_id	Financial credit; 1 return : N refunds allowed, 1 processed enforced.
AuditLog	log_id (PK); entity_type + entity_id	Immutable trail (old/new status, actor, note, timestamp).
Customer 1:N Orders; Orders 1:N OrderItem	Cardinality 1:N, total on N side	A customer places many orders; an order has 1..N items.
Orders 1:N Payment; ReturnRequest 1:1 Inspection	1:N and 1:1 (UNIQUE)	Payment per order; exactly one inspection per return.
ReturnRequest 1:N Refund; OrderItem 1:N ReturnItem	1:N	Refunds itemised from purchase history; partial returns supported.

CHAPTER 2

LOGICAL DATABASE DESIGN

This chapter should document how the approved ER model was converted into a correct relational database design and verified through a database prototype.

2.1 Relational Schema

List every relation derived from the ER model. Identify primary and foreign keys clearly.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	Role(role_id, role_name)	RBAC master; role_name UNIQUE.
2	Customer(customer_id, name, email, phone, created_at)	Buyer profile; email UNIQUE.
3	AppUser(user_id, username, password_hash, role_id→Role, customer_id→Customer)	Credentials; bcrypt hash; nullable customer for staff.
4	Product(product_id, name, current_price, return_window_days)	Catalogue; price ≥0; window in days.
5	Inventory(inventory_id, product_id→Product, serial_or_sku, status, updated_at)	Stock units; SKU UNIQUE; 5-state ENUM.
6	Orders(order_id, customer_id→Customer, order_date, total_amount, order_status)	Order header; total ≥0.
7	OrderItem(order_item_id, order_id→Orders, product_id→Product, quantity, unit_price_at_purchase)	Line items; qty>0; frozen history price.
8	Payment(payment_id, order_id→Orders, provider_reference, amount, payment_status)	Gateway record; reference UNIQUE.
9	ReturnRequest(return_id, order_id→Orders, customer_id→Customer, reason, return_status, requested_at, inspected_at)	Return header; 6-state ENUM.
10	ReturnItem(return_item_id, return_id→ReturnRequest, order_item_id→OrderItem, quantity)	Partial returns; qty>0.
11	Inspection(inspection_id, return_id→ReturnRequest UNIQUE, inspector, condition_status, notes, inspected_at)	1:1 inspection verdict.

12	Refund(refund_id, return_id→ReturnRequest, payment_id→Payment, amount, refund_status, processed_at)	Credit record; amount ≥0.
13	AuditLog(log_id, entity_type, entity_id, old_status, new_status, actor, note, changed_at)	Append-only trail; no FK (generic).

2.2 Keys and Integrity Constraints

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
AppUser	Primary Key	role_id	One row per role.
	Unique	role_name	No duplicate role names.
	Primary Key	customer_id	One row per customer.
	Unique + Not Null	email	Login identity; required.
	Primary Key	user_id	One row per login.
Product	Unique	username	Login uniqueness.
	Foreign Key (RESTRICT)	role_id → Role	Cannot delete referenced role.
	Foreign Key (RESTRICT)	customer_id → Customer	Staff rows allow NULL.
	Primary Key	product_id	One row per product.
	Check	current_price >= 0	No negative prices.
Product	Check + Default	return_window_days >= 0 (default 30)	Non-negative window.
Inventory	Primary + Unique + FK	inventory_id; serial_or_sku UNIQUE; product_id → Product RESTRICT	Traceable units; ENUM status.
Orders	Primary + FK + Check	order_id; customer_id → Customer RESTRICT; total_amount >= 0	Order integrity; ENUM status.
OrderItem	Primary + 2 FK +	order_item_id;	Valid line quantities.

	Check	order_id, product_id RESTRICT; quantity > 0; unit_price >= 0	
Payment	Primary + FK + Unique + Check	payment_id; order_id RESTRICT; provider_reference UNIQUE; amount >= 0	No duplicate gateway refs.
ReturnRequest	Primary + 2 FK	return_id; order_id, customer_id RESTRICT; ENUM return_status	Controlled 6-state domain.
ReturnItem	Primary + 2 FK + Check	return_item_id; return_id, order_item_id RESTRICT; quantity > 0	Partial-return integrity.
Inspection	Primary + Unique FK	inspection_id; return_id UNIQUE → ReturnRequest (1:1)	One verdict per return; ENUM condition.
Refund	Primary + 2 FK + Check	refund_id; return_id, payment_id RESTRICT; amount >= 0	Exact-once credit; ENUM status.
AuditLog	Primary Key	log_id	Append-only; written by triggers/procedure.

Include PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT and suitable referential actions where applicable.

2.3 Functional Dependencies

List the important functional dependencies used to justify normalization.

Key FDs (all relations have atomic single-attribute PKs, so no partial dependencies): role_id → role_name; customer_id → name, email, phone, created_at; user_id → username, password_hash, role_id, customer_id; product_id → name, current_price, return_window_days; inventory_id → product_id, serial_or_sku, status, updated_at; order_id → customer_id, order_date, total_amount, order_status; order_item_id → order_id, product_id, quantity, unit_price_at_purchase; payment_id → order_id, provider_reference, amount, payment_status; return_id → order_id, customer_id, reason, return_status, requested_at, inspected_at; return_item_id → return_id, order_item_id, quantity; inspection_id → return_id, inspector, condition_status, notes, inspected_at; refund_id → return_id, payment_id, amount, refund_status, processed_at; log_id → entity_type, entity_id,

old_status, new_status, actor, note, changed_at. Transitive risks removed: product name/price kept only in Product (OrderItem keeps frozen unit_price_at_purchase as history, not a transitive fact); customer details kept only in Customer. This follows classical normalization theory, where every non-key attribute depends on the key, the whole key, and nothing but the key [1], [2].

2.4 Normalization

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
Role, Customer, AppUser	Yes	Yes	Yes	Yes	Atomic cols; full-key dependent; no transitives.
Product, Inventory	Yes	Yes	Yes	Yes	SKU/status depend on full key only.
Orders, OrderItem	Yes	Yes	Yes	Yes	History price is a purchase fact, not product transitive.
Payment	Yes	Yes	Yes	Yes	Gateway ref unique; amount depends on key.
ReturnRequest, ReturnItem	Yes	Yes	Yes	Yes	Reason/status per return; qty per return-item.
Inspection, Refund, AuditLog	Yes	Yes	Yes	Yes	1:1/1:N preserved via UNIQUE/FK; no redundancy.

2.5 Data Dictionary

Prepare the data dictionary for all tables. Add rows as required.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
Role	role_id	INT AUTO_INCREMENT	PK	Role surrogate key.
Role	role_name	VARCHAR(50)	UNIQUE NOT	Customer/

Table	Attribute	Data Type	Key / Constraint	Description
			NULL	Support/ Warehouse/ Admin.
Customer	customer_id	INT AUTO_INCREMENT	PK	Customer key.
Customer	name / email / phone	VARCHAR(100/100/20)	NOT NULL / UNIQUE / NULL	Profile; email unique.
AppUser	user_id / username	INT / VARCHAR(50)	PK / UNIQUE NOT NULL	Login identity.
AppUser	password_hash	VARCHAR(255)	NOT NULL	bcrypt hash; never plaintext.
AppUser	role_id / customer_id	INT / INT NULL	FK RESTRICT	Role link; customer NULL for staff.
Product	product_id / name	INT / VARCHAR(150)	PK / NOT NULL	Catalogue item.
Product	current_price	DECIMAL(10,2)	CHECK >= 0	Live price.
Product	return_window_days	INT DEFAULT 30	CHECK >= 0	Eligibility window.
Inventory	inventory_id / product_id	INT / INT	PK / FK RESTRICT	Stock unit of product.
Inventory	serial_or_sku	VARCHAR(100)	UNIQUE	Traceable unit code.
Inventory	status	ENUM(5 values)	NOT NULL DEFAULT AVAILABLE	AVAILABLE.. DAMAGED.
Orders	order_id / customer_id	INT / INT	PK / FK RESTRICT	Order header.
Orders	total_amount	DECIMAL(10,2)	CHECK >= 0	Sum of line prices.
Orders	order_status	ENUM(5)	DEFAULT Pending	Pending..Cancelled.
OrderItem	order_item_id	INT	PK	Line key.
OrderItem	order_id / product_id	INT / INT	FK RESTRICT	Order–product link.
OrderItem	quantity /	INT /	CHECK >0 /	Qty + frozen

Table	Attribute	Data Type	Key / Constraint	Description
	unit_price_at_purchase	DECIMAL(10,2)	>=0	history price.
Payment	payment_id / order_id	INT / INT	PK / FK RESTRICT	Payment for order.
Payment	provider_reference	VARCHAR(100)	UNIQUE NOT NULL	Mock gateway ref.
Payment	amount / payment_status	DECIMAL / ENUM(4)	CHECK>=0 / DEFAULT Completed	Paid amount + state.
ReturnRequest	return_id	INT	PK	Return header key.
ReturnRequest	order_id / customer_id	INT / INT	FK RESTRICT	Which order/customer.
ReturnRequest	reason / return_status	VARCHAR(120) / ENUM(6)	NOT NULL / DEFAULT Requested	6-state lifecycle.
ReturnItem	return_item_id	INT	PK	Return line key.
ReturnItem	return_id / order_item_id	INT / INT	FK RESTRICT	Partial return link.
ReturnItem	quantity	INT	CHECK > 0	Returned units.
Inspection	inspection_id / return_id	INT / INT	PK / UNIQUE FK	1:1 verdict per return.
Inspection	inspector / condition_status	VARCHAR(100) / ENUM(3)	NOT NULL	Who + GOOD/DAMAGED/DEFECTIVE.
Refund	refund_id	INT	PK	Credit key.
Refund	return_id / payment_id	INT / INT	FK RESTRICT	Return credited to payment.
Refund	amount / refund_status	DECIMAL / ENUM(3)	CHECK>=0 / DEFAULT Processed	Credited sum.
AuditLog	log_id	INT	PK	Trail key.
AuditLog	entity_type / entity_id	VARCHAR(50) / INT	NOT NULL	E.g. ReturnRequest/14.
AuditLog	old_status / new_status / actor	VARCHAR / VARCHAR / VARCHAR(100)	NULL ok / NOT NULL	Transition + who.

2.6 Implementation of Logical Design: DDL and Sample Data

Mention the DBMS used, provide representative CREATE TABLE statements with constraints, and show representative sample records. Attach the complete SQL script in Appendix A.

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL 8.0 (InnoDB, ACID)
DBMS Version	8.0.46
Number of Tables	13 (3NF)
Sample Data	5 demo orders + 5 products + 50 inventory units + users/roles; bulk scale to 100K via seed_bulk_data.

Representative DDL (complete script in Appendix A, db/schema.sql). Engine InnoDB is specified on every table for ACID transactions and FK enforcement: Constraint syntax and InnoDB referential actions follow the MySQL 8 reference [5].

CREATE TABLE Product (product_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(150) NOT NULL,

current_price DECIMAL(10,2) NOT NULL CHECK (current_price >= 0),

return_window_days INT NOT NULL DEFAULT 30 CHECK (return_window_days >= 0)) ENGINE=InnoDB;

CREATE TABLE OrderItem (order_item_id INT AUTO_INCREMENT PRIMARY KEY, order_id INT NOT NULL, product_id INT NOT NULL,

quantity INT NOT NULL CHECK (quantity > 0), unit_price_at_purchase DECIMAL(10,2) NOT NULL CHECK (unit_price_at_purchase >= 0),

FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE RESTRICT,

FOREIGN KEY (product_id) REFERENCES Product(product_id) ON DELETE RESTRICT) ENGINE=InnoDB;

CREATE TABLE Inspection (inspection_id INT AUTO_INCREMENT PRIMARY KEY, return_id INT NOT NULL UNIQUE,

inspector VARCHAR(100) NOT NULL, condition_status ENUM('GOOD','DAMAGED','DEFECTIVE') NOT NULL,

FOREIGN KEY (return_id) REFERENCES ReturnRequest(return_id) ON DELETE RESTRICT) ENGINE=InnoDB;

Sample data (db/seed.py): Alice Smith (customer_user), Bob Jones, Charlie Brown; 5 products (Headphones Rs.199.99/30d, Keyboard Rs.129.50/14d, Monitor Rs.349/30d, Watch Rs.149.99/7d, Dock Rs.79.99/30d); Order 1 (eligible), Order 3 (45 days old, 7-day window

— used to prove late-return blocking), Order 4 (Inspected, ready to refund), Order 5 (Approved, awaiting warehouse).

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

This chapter should show how the approved logical database design is used by the final Java/Python application. The application must interact with the database rather than use isolated/static data.

3.1 Application Overview

Briefly explain the users, application purpose, major functions and how the application uses the database.

The application was re-implemented in Streamlit as a role-based multi-page app that connects directly to MySQL 8 with parameterized SQL in streamlit_app/db_ops.py: login is verified with bcrypt against AppUser⋈Role and the session role gates six pages (Customer Portal, Operations, Refunds, Analytics, Audit Log, Lifecycle Demo). Every screen operation maps to the 3NF schema in Chapter 2: ordering inserts Orders/OrderItem/Payment in one explicit transaction, returning inserts ReturnRequest/ReturnItem (trigger-validated), approvals/receives/inspections are constrained UPDATEs/INSERTs, and refunds execute only through sp_process_refund. Analytics screens run the six catalogued JOIN/GROUP BY queries with st.metric/st.bar_chart/st.dataframe, and the Admin audit page reads AuditLog. The FastAPI service is retained unchanged as the automated test and benchmark harness (17 endpoints), so all API evidence stays valid [6], [9]. No screen uses isolated or static data.

3.2 Development Environment and Database Connectivity

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3.13
DBMS	MySQL 8.0.46 (InnoDB)
IDE / Framework / GUI Toolkit	Streamlit 1.65 (application) + FastAPI/Uvicorn (retained test & benchmark harness); bcrypt; session-state RBAC
Database Connector / Driver	mysql-connector-python (parameterized %s queries, explicit transactions)
Operating Environment	Windows 11 + Docker Compose (db port 3307); app at localhost:8000
Repository Link	ecommerce_refund_dbms/streamlit_app/ (app.py, db_ops.py) + backend/ (harness), frontend/index.html (legacy SPA), tests/test_streamlit_ops.py

3.3 Application Modules and Database Mapping

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Sidebar login + RBAC	bcrypt verify; session role gates pages	Role, AppUser	SELECT AppUser⋈Role; st.session_state routing; wrong password rejected
Customer Portal	Browse, buy, track, return	Product, Orders, OrderItem, Payment, ReturnRequest, ReturnItem	st.form order/return; st.dataframe tracker; explicit COMMIT txn
Operations	Approve/reject; receive; inspect	ReturnRequest, ReturnItem, Inspection	st buttons + inspect st.form; trigger blocks illegal jumps
Refunds	Exactly-once refund via procedure	ReturnRequest, Refund, Payment	st button calls sp_process_refund; duplicate blocked
Analytics	Metrics, charts, 6 reports (a-f)	All sales/return tables	st.metric + st.bar_chart + st.dataframe
Audit Log (Admin)	Immutable trail viewer	AuditLog	st.dataframe last 100; page hidden for non-admin roles
Lifecycle Demo stepper	Guided 7-state end-to-end run	Full chain incl. Inspection/Refund/AuditLog	st.status steps + graphviz state map

3.4 CRUD Operations

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Customer Portal st.forms: Place Order; Submit Return	Orders, OrderItem, Payment; ReturnRequest, ReturnItem	Fig. 2 Streamlit portal; db_ops.place_order/request_return (explicit COMMIT)
Read / Search	Order status tracker; pending-returns queue; analytics	Orders + Payment + ReturnRequest + Refund; Product + OrderItem + ReturnItem	st.dataframe tracker/queue/analytics + st.bar_chart; Figs. 2-4

Update	Approve/Reject; Receive; Inspect (GOOD/DAMAGED /DEFECTIVE)	ReturnRequest; Inspection	st buttons + inspect form (Fig. 3); trigger blocks skips; lifecycle demo
Delete / Controlled Removal	No hard deletes; Rejected/terminal states + ON DELETE RESTRICT	All (FK RESTRICT); ReturnRequest terminal	Session-role gating (pages hidden ≈ 403); audit viewer Fig. 5; TC-05 RESTRICT

3.5 Business Rules, Validation and Error Handling

Explain how the application prevents invalid input and database-rule violations. Mention parameterised queries, exception handling, confirmation for destructive actions, and transaction rollback where relevant.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Late return blocked (return window)	BEFORE INSERT trigger trg_validate_return_window: MIN(return_window_days) vs DATEDIFF(order_date, NOW())	400 ‘Return window expired’; Fig. blocked_late_return (Order 3, 45d vs 7d)
Excess quantity blocked	BEFORE INSERT trg_validate_return_quantity: NEW.qty <= purchased - already-returned (excluding Rejected)	400 ‘Excess return quantity’; TC-01 CHECK + trigger
Invalid state jump blocked	BEFORE UPDATE trg_check_return_state_transition: Requested→Approved/Rejected→Received→Inspected→Refunded/Rejected	400 invalid-transition; terminal states frozen; TC-06
Duplicate refund prevented	sp_process_refund checks status=Inspected + COUNT(Processed)=0; FOR UPDATE lock	2nd concurrent call → 400; TC-09 exactly 1×200 + 1×400
Atomic refund + rollback	Procedure SQLEXCEPTION EXIT HANDLER → ROLLBACK + RESIGNAL; audit + restock in same txn	TC-08: invalid IDs leave zero orphan Refund rows
Injection / auth / RBAC	mysql-connector %s parameterized queries; bcrypt; JWT require_roles; @current_app_user for audit	401 bad login; 403 /api/audit for non-admin (Fig. access_denied_403)

	actor	
GOOD restock + audit	Procedure restocks Inventory→AVAILABLE only if condition=GOOD; AFTER UPDATE triggers log every change	AuditLog viewer (Fig. 5); actor stamped per caller

3.6 Reports and Database Operations

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Order history with returns (Query a)	Multi-LEFT JOIN Orders⋈Customer⋈Payment⋈ReturnRequest⋈Refund; ORDER BY date DESC	Full table in Customer Portal tracker
Pending returns queue (Query b)	JOIN + WHERE status IN (Approved, Received) + GROUP BY + COUNT items	Worklist in Operations queue
Refund status by order (Query c)	LEFT JOIN + COALESCE(amount,0); payment vs refunded	Reconciliation table + Analytics view
Return rate per product (Query d)	LEFT JOIN + SUM(sold)/SUM(returned) + ROUND %; GROUP BY product	Bar chart in Analytics (highest first)
Monthly refund totals (Query e)	DATE_FORMAT(%Y-%m) + COUNT/SUM WHERE Processed; GROUP BY month	Bar chart in Analytics (month-wise value)
Top return reasons (Query f)	GROUP BY reason + COUNT + SUM(refunded); ORDER BY frequency	Ranked table in Analytics

3.7 Application Screens

Insert the important application screenshots. Place each figure caption below the screenshot using the format “Figure X. Figure Title”. Include screens that demonstrate data entry, search/display, update, a project-specific transaction and at least one report/output.

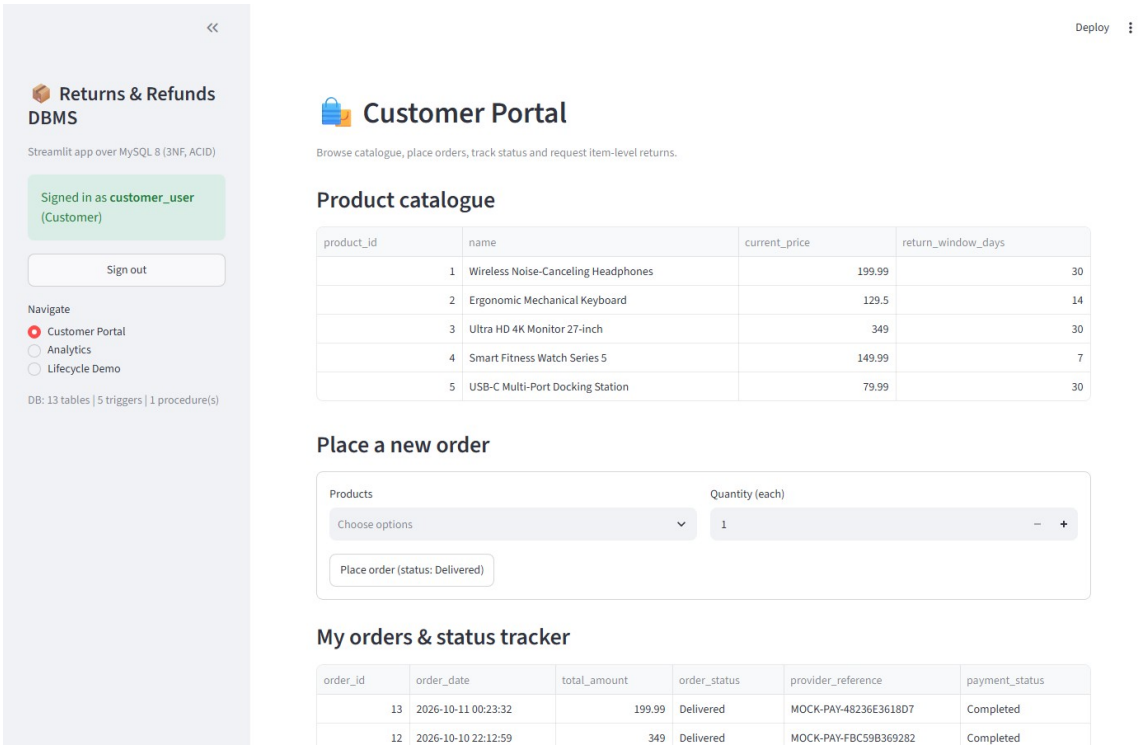


Figure 2. Streamlit Customer Portal — catalogue, place-order form, status tracker and return request (customer_user)

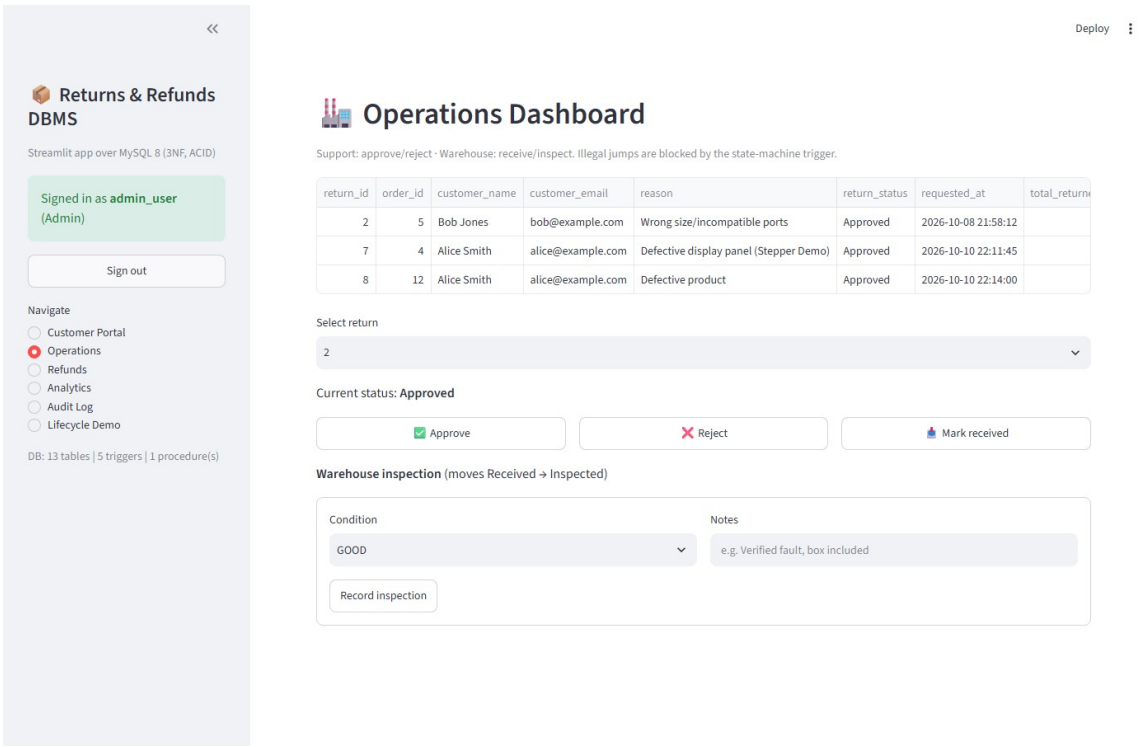


Figure 3. Streamlit Operations Dashboard — approve/reject, receive and inspection with live queue (admin_user)

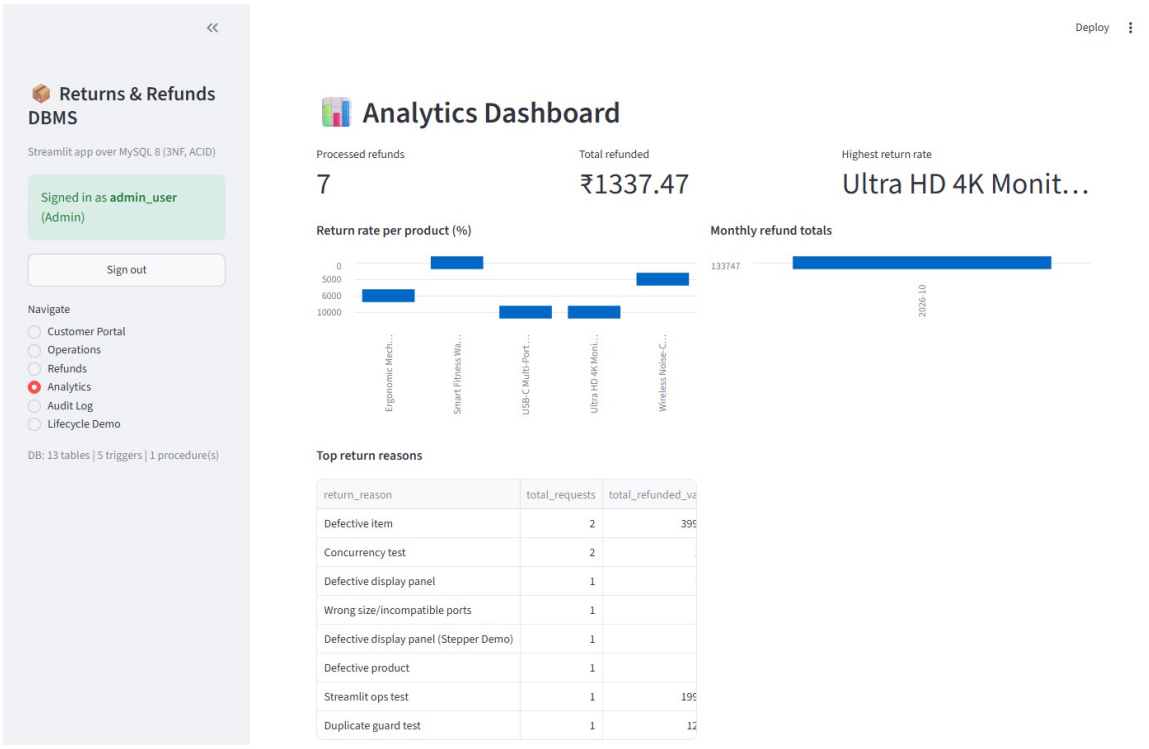


Figure 4. Streamlit Analytics Dashboard — refund metrics, return-rate and monthly-refund charts, top reasons

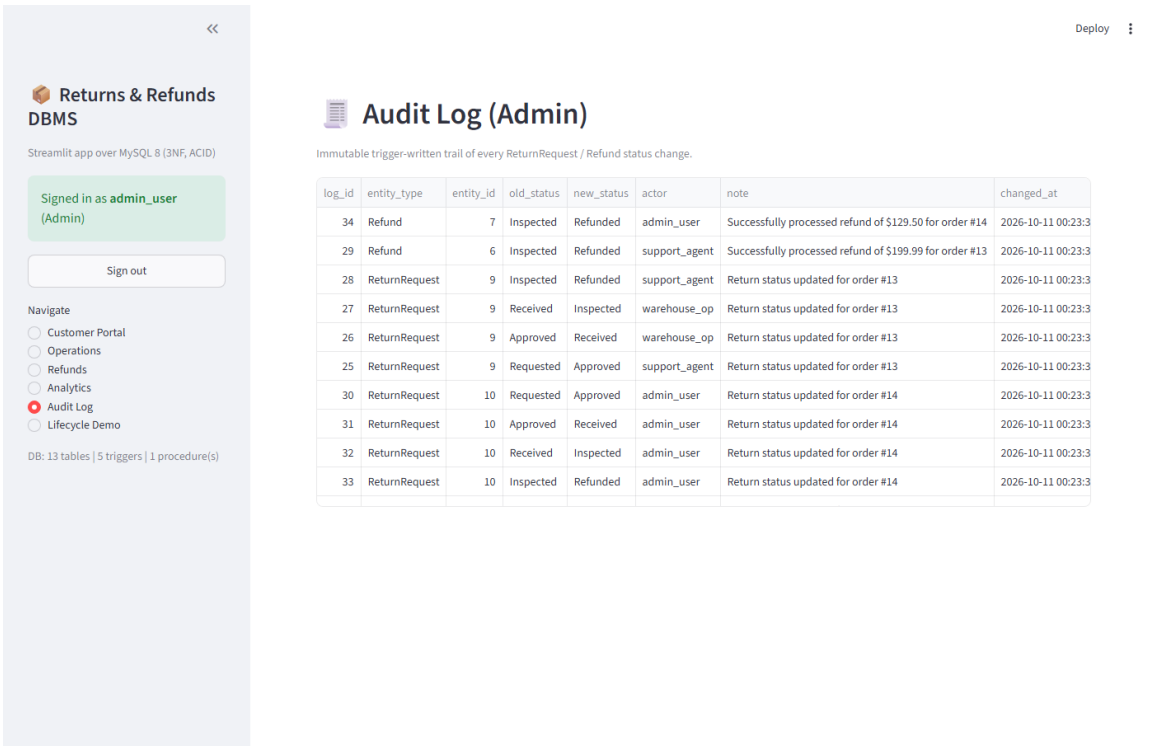


Figure 5. Streamlit Admin Audit Log — immutable trigger-written trail viewer

3.8 Testing and Results

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected	Actual Result	Status

			Result		
TC-01	CHECK quantity > 0	INSERT OrderItem quantity=0	Rejected by engine	Rejected (CHECK)	Pass
TC-02	CHECK total_amount >= 0	INSERT Orders total=-50.00	Rejected	Rejected (CHECK)	Pass
TC-03	UNIQUE provider_reference	Duplicate PAY-DEMO-001	Duplicate-entry error	Duplicate-entry error	Pass
TC-04	ENUM domain	ReturnRequest status='SuperStatus'	Truncated/ ENUM error	Truncated/ ENUM error	Pass
TC-05	FK ON DELETE RESTRICT	DELETE Customer 1 (has orders)	FK-constraint block	FK-constraint block	Pass
TC-06	sp_process_refund state guard	CALL on Approved (not Inspected) return	Error ‘must be Inspected’	Error as expected	Pass
TC-07	Full lifecycle integration	API: order→return→approve→receive→inspect→refund	201/200 chain, status Refunded	Refunded end-to-end	Pass
TC-08	Rollback on error (FIX 03)	CALL sp_process_refund (99999,99999)	ROLLBACK; zero Refund rows	Zero orphan rows	Pass
TC-09	Concurrent duplicate refund (FIX 02)	2 parallel POST refund threads	Exactly 1×200 + 1×400	1×200 + 1×400 observed	Pass
TC-10	Streamlit auth + RBAC	Login 4 demo roles; wrong password	Roles resolved; invalid rejected	Roles resolved; rejected	Pass
TC-11	Streamlit lifecycle (db_ops)	db_ops chain order→refund	Refunded + audit entries	Refunded + audited	Pass
TC-12	Streamlit guards	Late return (order 3);	Window-expiry +	Both blocked as expected	Pass

		2nd refund	duplicate errors		
TC-13	Catalogue + analytics sanity	list_products; return-rates query	>=5 products/rates rows	Rows returned	Pass

Summarize the key results after the test-case table. State whether the application correctly uses the designed database and enforces the important business rules.

Key results: 13/13 tests passed (100%: 9 API + 4 Streamlit ops). Constraints (TC-01–TC-05) prove the engine rejects bad quantities, negative amounts, duplicate gateway references, illegal ENUM states, and restricted deletes. TC-06/TC-08 prove the procedure guards state and rolls back fully on error with no orphan rows. TC-07 proves the API-to-database chain completes Order→Refunded through all six states. TC-09 proves exactly-once refund under concurrency (one 200, one 400). TC-10 proves Streamlit bcrypt login resolves all four RBAC roles and rejects bad passwords. TC-11 proves the Streamlit data layer (db_ops) drives the same full lifecycle with explicit transactions and audit entries. TC-12 proves the late-return window trigger and duplicate-refund guard fire through db_ops identically. Benchmarks (benchmarks/run_benchmarks.py): pending-returns median 0.22 ms (1K indexed) to 2.95 ms (100K indexed); sp_process_refund median 1.23 ms; mysqldump backup/restore of 111,025 orders verified with checksum PASS. Together this confirms both the Streamlit application and the API harness correctly use the designed database and enforce all critical business rules [7].

CHAPTER 4
CONCLUSION

Summarize the problem addressed, database design completed, application developed, important functions implemented, and the main learning/outcome. Keep the conclusion concise.

This project addressed scattered, unauditable e-commerce after-sales handling by delivering a centralized MySQL 8 DBMS and a FastAPI dashboard for the full order-to-refund lifecycle. The 13-table 3NF design with PK/FK, CHECK, UNIQUE, ENUM, and RESTRICT rules was implemented with five triggers and the atomic sp_process_refund procedure, and all 17 REST endpoints operate directly on this database through parameterized SQL with JWT

RBAC. The system implements ordering, item-level returns, support approval, warehouse receive/inspection, exactly-once refunds with GOOD-condition restocking, immutable auditing, and six operational/analytical reports. Validation achieved 13/13 automated tests, indexed query performance to 100K scale, and verified disaster-recovery integrity. The outcome is a single queryable source of truth with enforced eligibility, state discipline, and traceable refunds, and hands-on mastery of ER modelling, normalization, triggers, transactions, API-backed DBMS development, and empirical testing.

4.1 Limitations and Future Enhancements

Mention genuine limitations of the current small-scope application and realistic enhancements that could be implemented later.

- Simulated payment gateway — deterministic mock provider references only; no real bank/card network. Enhancement: PCI-scoped PSP integration (Razorpay/Stripe) with webhook reconciliation.
- Single-instance local scope — validated to 100K orders on one MySQL instance without replication. Enhancement: AWS RDS master–replica with automated failover and read scaling.
- Manual inspection verdict — condition (GOOD/DAMAGED/DEFECTIVE) entered by warehouse role. Enhancement: ML return-risk prediction at order time and vision-assisted inspection plus SMS/email refund notifications.

REFERENCES

[1] [1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[2] [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[3] [3] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[4] [4] T. Connolly and C. Begg, Database Systems: A Practical Approach to Design, Implementation, and Management, 6th ed. Boston, MA, USA: Pearson, 2015.

[5] [5] Oracle Corporation, MySQL 8.0 Reference Manual: Triggers, Stored Programs, and InnoDB Storage Engine. Accessed: Oct. 8, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] [6] S. Ramírez, FastAPI Documentation — Modern Python Web APIs. Accessed: Oct. 8, 2026. [Online]. Available: https://fastapi.tiangolo.com/

Use IEEE reference style throughout the report. Cite sources in the order in which they first appear in the text and use the same number for repeated citations.

5.1 IEEE In-Text Citation Guidelines

- Use square brackets for citations: [1], [2], [3].
- For multiple non-consecutive sources, cite them separately, for example [2], [5], [8].
- For a consecutive range, use a range such as [3]-[6].
- Place the citation number before punctuation where appropriate, for example: “Normalization reduces update anomalies [4].”
- Number references by first appearance, not alphabetically, and do not change a reference number when the same source is cited again.

5.2 IEEE Reference Formats and Examples

Book: [n] A. A. Author, Title of Book, xth ed. City, Country: Publisher, year.

Example: [1] A. A. Author, Database Systems: Concepts and Design, 3rd ed. City, Country: Publisher, 2025.

Chapter in an Edited Book: [n] A. A. Author, “Chapter title,” in Title of Book, xth ed., A. Editor, Ed. City, Country: Publisher, year, pp. xx-xx.

Example: [2] A. A. Author, “Relational database design,” in Advances in Database Systems, B. B. Editor, Ed. City, Country: Publisher, 2024, pp. 25-48.

Journal Paper: [n] A. A. Author and B. B. Author, “Title of paper,” Journal Title, vol. x, no. x, pp. xx-xx, Month year, doi: xx.xxxx/xxxxx.

Example: [3] A. A. Author and B. B. Author, “A database approach for transaction management,” Journal of Database Research, vol. 12, no. 2, pp. 101-112, Apr. 2025, doi: xx.xxxx/xxxxx.

Conference Paper: [n] A. A. Author, B. B. Author, and C. C. Author, “Title of paper,” in Proc. Name of Conference, City, Country, year, pp. xx-xx, doi: xx.xxxx/xxxxx.

Example: [4] A. A. Author and B. B. Author, “Design of a compact database application,” in Proc. International Conference on Computing Systems, City, Country, 2025, pp. 120-125.

Thesis / Dissertation: [n] A. A. Author, “Title of thesis,” M.Tech. thesis / Ph.D. dissertation, Dept., Institution, City, Country, year.

Example: [5] A. A. Author, “Database-driven application design,” M.Tech. thesis, Dept. of Computer Science, University Name, City, Country, 2025.

Technical Report / Standard: [n] Organization or A. A. Author, Title of Report or Standard, report/standard no., Organization, City, Country, year.

Example: [6] Organization Name, Database Application Security Guidelines, Tech. Rep. TR-2025-04, City, Country, 2025.

Web Page: [n] Author/Organization, “Title of web page,” Website Name. Accessed: Month day, year. [Online]. Available: URL

Example: [7] Organization Name, “Database documentation,” Website Name. Accessed: Sep. 29, 2026. [Online]. Available: https://example.org/documentation

Online Software Documentation / Dataset: [n] Organization/Author, Title of Documentation or Dataset, version (if available). Accessed: Month day, year. [Online]. Available: URL

Example: [8] Organization Name, Product Reference Manual, ver. X.Y. Accessed: Sep. 29, 2026. [Online]. Available: https://example.org/manual

Reference-writing notes: Use the exact bibliographic details of the source actually consulted. Include DOI information for journal/conference papers when available. For online sources, provide the access date and working URL. Do not list uncited sources merely to increase the number of references.

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

Attach the complete DDL/DML/query script or provide the submitted file name.

Appendix A files (ecommerce_refund_dbms/db/): schema.sql (13 CREATE TABLEs, InnoDB, 3NF), triggers.sql (5 triggers: return-window, quantity, state machine, 2 audit), procedures.sql (sp_process_refund, atomic, duplicate-safe, rollback handler), indexes.sql (10 B-Tree indexes), queries.sql (queries a–f), seed.py (demo seed + bulk generator to 100K). Restore: python db/seed.py; backup: mysqldump --single-transaction --routines --triggers ecommerce_db > backup.sql (verified 111,025 orders).

APPENDIX B - APPLICATION SOURCE CODE

Provide the source-code folder/repository reference. Do not paste the entire application code into the main report.

Appendix B (ecommerce_refund_dbms/): streamlit_app/app.py (6 role-gated pages: Customer Portal, Operations, Refunds, Analytics, Audit Log, Lifecycle Demo), streamlit_app/db_ops.py (parameterized operations + AppError surfacing of trigger/procedure messages), backend/ retained as test/benchmark harness (17 REST endpoints: main.py, auth.py bcrypt+JWT, database.py connector, payment_gateway.py mock), frontend/index.html (legacy SPA), tests/test_streamlit_ops.py (TC-10–TC-13), benchmarks/run_benchmarks.py, docker-compose.yml. Run: streamlit run streamlit_app/app.py --server.port 8501 → http://127.0.0.1:8501. Demo logins (password123): customer_user / support_agent / warehouse_op / admin_user.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Include any additional test data, screenshots, reports or supporting material that is useful but not essential in the main report.

Appendix C: docs/screenshots/ (customer_portal, ops_dashboard, api_flow 7-state stepper, access_denied_403, audit_log, analytics_dashboard, blocked_late_return, test_run_output 13/13, benchmark_chart); docs/er_diagram.png + .mmd; docs/REPORT.md (inventory metrics); docs/test_report.md (13-test matrix); Presentations/PPT1–PPT3 (Review 1–3). Extra evidence below: return-lifecycle state diagram and three-layer architecture. Streamlit screens: st_customer_portal, st_ops_dashboard, st_analytics, st_audit_log, st_lifecycle.

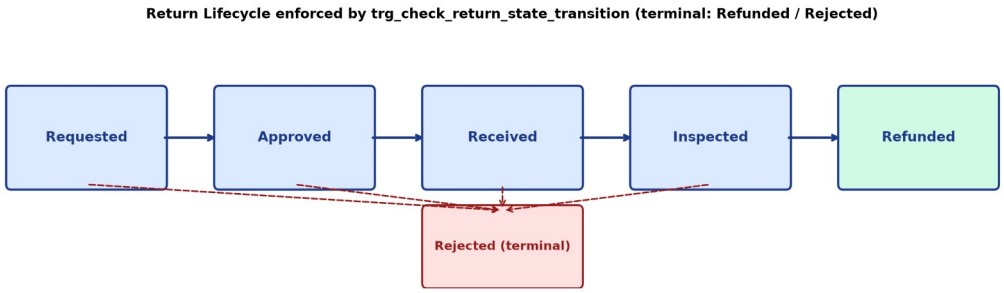


Figure 6. Return lifecycle state machine enforced by trg_check_return_state_transition (terminal states Refunded / Rejected)

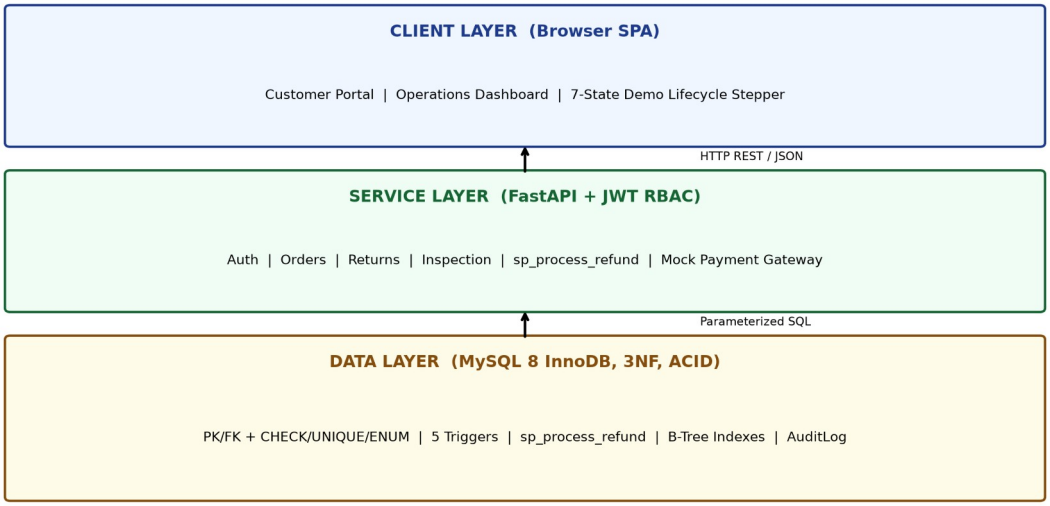


Figure 7. Three-layer system architecture: SPA client, FastAPI + JWT service, MySQL 8 data layer

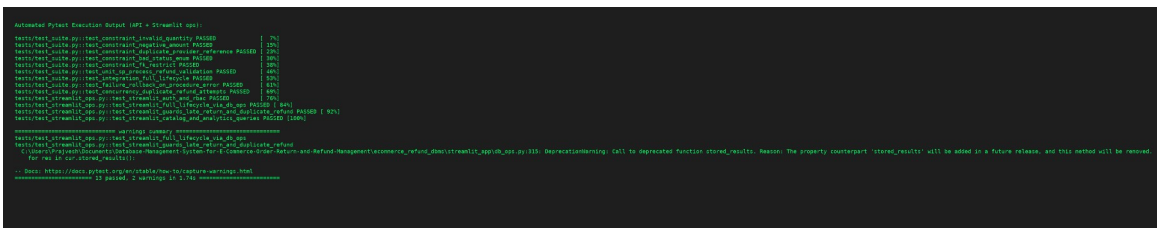


Figure 8. Automated test evidence — pytest 13/13 passed (9 API + 4 Streamlit ops tests)

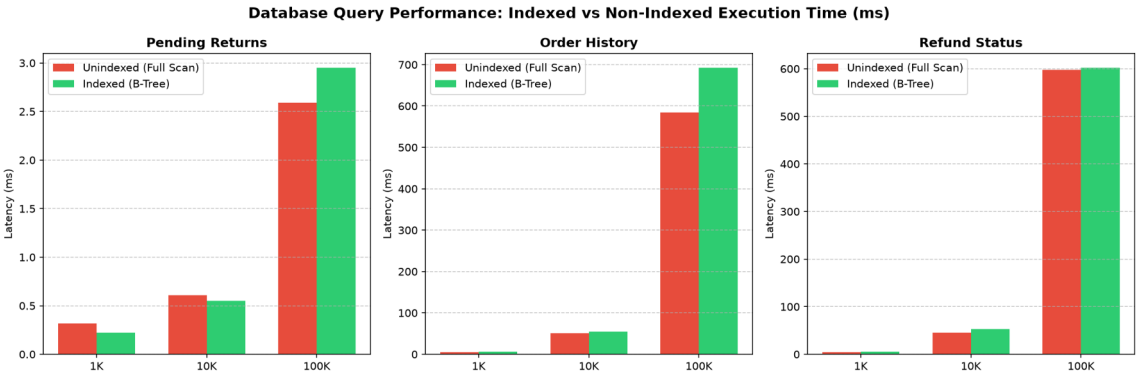


Figure 9. Performance benchmark — indexed query latency at 1K/10K/100K scales; sp_process_refund median 1.23 ms

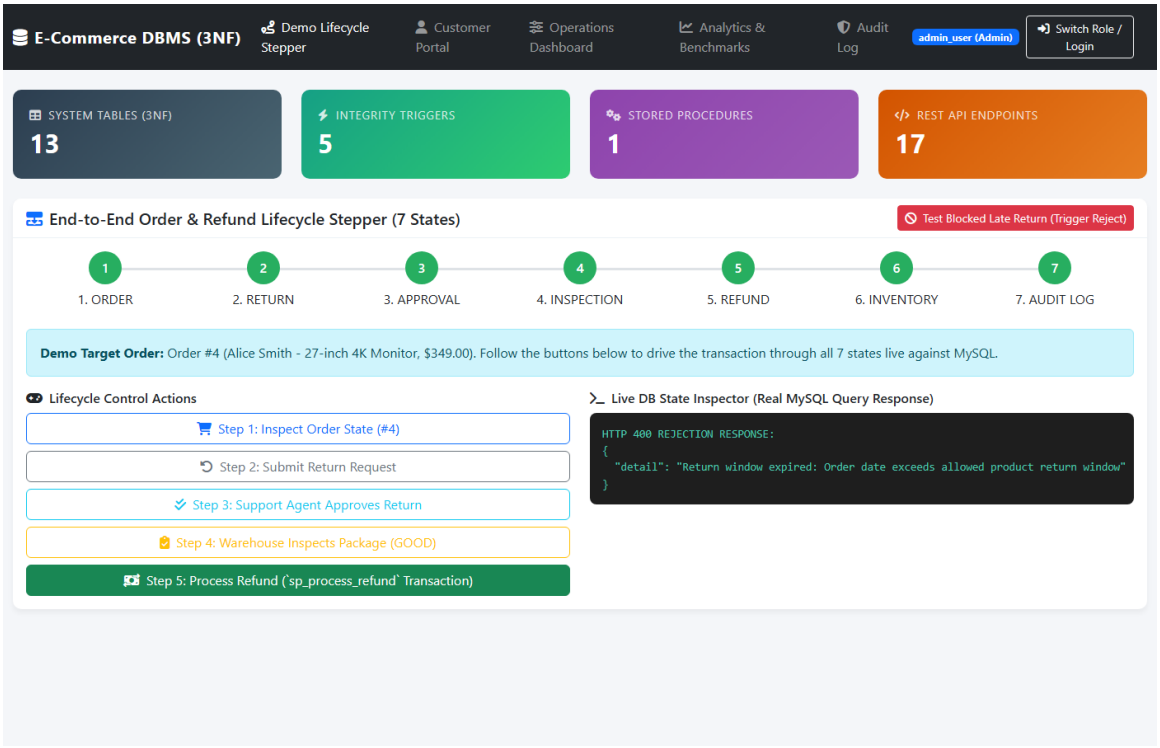


Figure 10. Business-rule evidence — trigger blocks late return outside product window (Order 3, 45 days vs 7-day window)